Phân loại trái cây tươi và hỏng bằng mạng tích chập sâu và học chuyển giao đa tầng trên đặc trưng hình thái

Fresh and Rotten Fruit Classification Using Deep Convolutional Networks and Multi-Stage Transfer Learning on Morphological Features

Hoàng Công Hoàn

Khoa Công nghệ Thông tin, Trường Đại học Công Nghệ Kỹ thuật Hưng Yên, Hưng Yên, Việt Nam

Mã sinh viên: 10123137

Abstract: Bài báo trình bày một hệ thống phân tích ảnh để phân loại tự động nông sản tươi và hỏng thành 6 lớp: táo, chuối, cam mỗi loại tươi hoặc hỏng, phục vụ kiểm soát chất lượng trong chuỗi cung ứng thực phẩm. Dữ liệu gồm 13.599 ảnh từ bộ dữ liệu công khai trên Kaggle [36], được chia độc lập thành 8.721 ảnh huấn luyện, 2.180 ảnh thẩm định và 2.698 ảnh kiểm thử. So sánh ba kiến trúc: CNN tự xây dựng với *Global Average Pooling* và *Batch Normalization*, MobileNetV2 và ResNet50 qua hai pha đối chứng: pha cơ sở (*Baseline*, backbone đóng băng) và pha nâng cao (*Advanced*, tăng cường dữ liệu, trọng số lớp, dừng sớm và tinh chỉnh hai bước). Quy trình được thiết kế để tránh ba lỗi phương pháp luận phổ biến: rò rỉ dữ liệu qua tăng cường, bùng nổ tham số do `Flatten`, và cập nhật sai thống kê BatchNorm khi tinh chỉnh. ResNet50 pha cơ sở đạt độ chính xác kiểm thử 99,74%, khoảng tin cậy Wilson 95%: 99,47–99,87% và Macro-F1 đạt 0,9973; MobileNetV2 đạt 99,67% với số tham số ít hơn khoảng 10 lần và suy luận nhanh gấp khoảng 3 lần trên CPU (14 ms/ảnh). Hai mô hình này không khác biệt có ý nghĩa thống kê trên tập kiểm thử. Trong 7 ảnh bị phân loại sai của ResNet50, chỉ 4 ảnh là lỗi tươi/hỏng, và không có quả tươi nào bị loại nhầm. Hệ thống được đóng gói thành ứng dụng web với Flask API và giao diện HTML/CSS/JS.

Từ khóa: phân loại ảnh, học sâu, học chuyển giao, ResNet50, chất lượng nông sản, rò rỉ dữ liệu, khoảng tin cậy Wilson.

I. GIỚI THIỆU

Theo báo cáo của Tổ chức Lương thực và Nông nghiệp Liên Hợp Quốc (FAO) [1], khoảng một phần ba lượng thực phẩm sản xuất cho con người bị thất thoát mỗi năm, trong đó rau quả thuộc nhóm có tỷ lệ hao hụt cao nhất. Nguyên nhân trọng yếu là khâu kiểm soát chất lượng sau thu hoạch còn chậm, thủ công và mang tính chủ quan [2]. Thị giác máy tính và học sâu mở ra triển vọng tự động hóa việc phân loại quả tươi và quả hỏng ngay trên băng chuyền hoặc trong kho bảo quản lạnh [3, 4].

Tuy nhiên, bài toán phân loại độ tươi/hỏng trong điều kiện thực tế đối mặt với bốn thách thức kỹ thuật lớn:

1. **Tính đa dạng của tổn thương bề mặt:** Khác với nhận dạng loài vốn dựa trên hình dạng tương đối cố định, quả hỏng biểu hiện qua đốm mốc vi sinh, vết thâm tím, nhăn cục bộ hoặc đổi sắc không đồng đều. Mô hình nếu chỉ học màu sắc tổng thể rất dễ nhầm quả chín tự nhiên với quả bắt đầu phân hủy.
2. **Mất cân bằng dữ liệu tự nhiên:** Tỷ lệ quả hỏng và quả tươi giữa các loại thường không đồng đều, khiến mô hình thiên lệch về lớp đa số nếu hàm mất mát không được bù trọng số [5].
3. **Lỗi phương pháp luận phổ biến:** các lỗi thường gặp như áp dụng tăng cường ngẫu nhiên lên cả tập thẩm định/kiểm thử, làm phẳng tensor bằng `Flatten` gây bùng nổ tham số và tinh chỉnh trực tiếp mạng tiền huấn luyện khi tầng đầu phân loại còn ngẫu nhiên, khiến gradient lớn phá hủy đặc trưng ImageNet [6, 7]. Rò rỉ thông tin giữa các tập dữ liệu còn làm kết quả lạc quan giả tạo [8].
4. **Ràng buộc triển khai trên phần cứng biên:** Hệ thống công nghiệp cần độ trễ suy luận thấp và bộ nhớ nhỏ để chạy trên CPU phổ thông, nhanh chóng tích hợp vào hệ thống sản xuất và kinh doanh.

Các ứng dụng bài báo hướng đến:

- **Ứng dụng vào hệ thống phân loại nông sản thông minh:** Xây dựng quy trình xử lý dữ liệu tự động toàn diện với pipeline `tf.data` [37] tách biệt nghiêm ngặt, giúp hệ thống nhận diện, sàng lọc và phân loại chính xác chất lượng nông sản ngoài thực tế sản xuất mà hoàn toàn không bị hiện tượng rò rỉ dữ liệu, đảm bảo độ khách quan tối đa cho mô hình.
- **Tối ưu hóa kiến trúc mạng:** Cải tiến mạnh mẽ mô hình bằng việc tích hợp lớp `GlobalAveragePooling2D` [31] kết hợp với `BatchNormalization` cùng cơ chế cô lập backbone [9], giúp mô hình duy trì trạng thái hoạt động ổn định, giảm thiểu tối đa tài nguyên tính toán khi đưa vào vận hành thực tế tại các nhà máy.
- **Kiểm định độ tin cậy môi trường công nghiệp:** Cung cấp hệ thống đánh giá thực tiễn toàn diện thông qua phân tích ma trận nhầm lẫn chi tiết, các chỉ số đo lường chuẩn xác như Precision, Recall, Macro-F1 [10, 11] và khoảng tin cậy Wilson 95% [12] nhằm cam kết độ chính xác tuyệt đối trước khi đưa dây chuyền vào thương mại hóa.
- **Quản trị rủi ro và bài toán kinh tế nông nghiệp:** Thực hiện phân tích lỗi chuyên sâu dựa trên hậu quả nghiệp vụ thực tế, đánh giá chi tiết tỷ lệ lọt quả hỏng so với việc loại nhầm quả tươi nhằm giảm thiểu tối đa thiệt hại kinh tế, tối ưu hóa lợi nhuận cho các cơ sở sản xuất, chế biến và đóng gói nông sản.
- **Triển khai thực tế trên phần cứng và Web:** Thực hiện đóng gói mô hình học sâu hoàn chỉnh dưới dạng trọng số nhẹ, tích hợp các kỹ thuật lượng tử hóa tiên tiến để triển khai thành công trên bo mạch nhúng Raspberry Pi thực tế và tích hợp mượt mà vào giao diện Web demo, phục vụ trực tiếp cho nhu cầu giám sát và vận hành của các nhà máy, hợp tác xã nông nghiệp hiện đại.

II. CÁC NGHIÊN CỨU LIÊN QUAN

A. Thị giác máy tính trong đánh giá chất lượng nông sản

Các khảo sát tổng quan [2, 3, 4] cho thấy tiếp cận từ các đặc trưng thủ công kết hợp bộ phân loại truyền thống sang mạng tích chập sâu. Bộ dữ liệu Fruits-360 [13] được sử dụng rộng rãi cho nhận dạng chủng loại quả. Bài toán nhận diện tươi hỏng phức tạp hơn ở chỗ dấu hiệu phân biệt nằm ở các tổn thương cục bộ nhỏ trên cùng một loài, đòi hỏi biểu diễn không gian chi tiết hơn và thiết kế tăng cường dữ liệu.

B. Kiến trúc CNN và học chuyển giao

AlexNet [14] mở đầu kỷ nguyên CNN sâu trên ImageNet [15, 16]; VGG [17] nhấn mạnh vai trò của việc chồng các bộ lọc 3×3 theo chiều sâu; ResNet [18] giải quyết hiện tượng suy biến khi mạng sâu nhờ kết nối tắt (shortcut connection); MobileNet [19] và MobileNetV2 [20] tối ưu hóa cho thiết bị biên thông qua tích chập tách chiều sâu; trong khi EfficientNet [21] đề xuất quy tắc mở rộng quy mô mạng có hệ thống. Về học chuyển giao, Yosinski và cộng sự [6] chỉ ra đặc trưng ở các tầng đầu mang tính tổng quát còn tầng sâu mang tính đặc thù tác vụ; Pan và Yang [22] hệ thống hóa các kịch bản chuyển giao tri thức; Kornblith và cộng sự [23] chứng minh các mô hình có độ chính xác cao hơn trên ImageNet thường chuyển giao hiệu quả hơn sang miền dữ liệu mới.

C. Mất cân bằng lớp, đánh giá và rò rỉ dữ liệu

Buda và cộng sự [5] khảo sát có hệ thống ảnh hưởng của mất cân bằng lớp lên mạng CNN. Các giải pháp ở cấp độ hàm mất mát gồm trọng số lớp nghịch đảo tần suất, class-balanced loss [24] và focal loss [25]. Khi phân bố dữ liệu lệch, chỉ số Accuracy có thể che giấu sự suy giảm ở lớp thiểu số, vì vậy cần báo cáo đồng thời Precision, Recall và Macro-F1 [26, 27]. Rò rỉ dữ liệu [8], thiên lệch bộ dữ liệu [28] và học tắt (shortcut learning) [29] là những nguyên nhân chính khiến độ chính xác trên tập công khai không phản ánh đúng hiệu năng thực địa; Raschka [30] đã tổng hợp các khuyến nghị chuẩn mực về đánh giá và lựa chọn mô hình.

III. PHƯƠNG PHÁP NGHIÊN CỨU

A. Phát biểu bài toán và Hàm mục tiêu

Đây là bài toán phân loại ảnh đơn nhãn đa lớp. Cho tập dữ liệu thực nghiệm gồm N mẫu quan sát:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \tag{1}$$

với $\mathbf{x}_i \in \mathbb{R}^{H \times W \times C}$ ($H = 224, W = 224, C = 3$) là tensor ảnh màu RGB và $y_i \in \{0, 1, \ldots, K-1\}$ là nhãn số nguyên của 6 phân lớp: `freshapples`, `freshbanana`, `freshoranges`, `rottenapples`, `rottenbanana`, `rottenoranges`.

Mục tiêu của mô hình là học ánh xạ tham số:

$$f_{\boldsymbol{\theta}} : \mathbb{R}^{H \times W \times C} \to \Delta^{K-1}, \quad \widehat{\mathbf{p}} = f_{\boldsymbol{\theta}}(\mathbf{x}) \tag{2}$$

Quá trình trích xuất đặc trưng và suy luận được thực hiện tuần tự qua các tầng ẩn nhằm ánh xạ không gian ảnh phức tạp thành vector điểm số thô. Vector xác suất dự đoán $\widehat{\mathbf{p}} = [\widehat{p}_0, \widehat{p}_1, \ldots, \widehat{p}_{K-1}]^T$, trong đó tổng các thành phần luôn thỏa mãn điều kiện chuẩn hóa $\sum_{k=0}^{K-1} \widehat{p}_k = 1$, được tính thông qua hàm kích hoạt Softmax chuẩn hóa nhằm chuyển đổi các giá trị đầu ra chưa ràng buộc thành phân phối xác suất tương ứng [7]:

$$\widehat{p}_k = \frac{\exp(z_k)}{\sum_{j=0}^{K-1} \exp(z_j)}, \quad \forall k \in \{0, 1, \ldots, K-1\} \tag{3}$$

trong đó z_k là giá trị logit của lớp k. Hàm mất mát Entropy chéo có trọng số [5] tính trên một lô xử lý $B = 32$ ảnh được định nghĩa độc lập như sau:

$$\mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{B} \sum_{i=1}^{B} w_{y_i} \log(\widehat{p}_{i,y_i}) \tag{4}$$

Hệ số trọng số phạt w_k tỷ lệ nghịch với tần suất xuất hiện của lớp k trong tập huấn luyện nhằm cân bằng biên độ gradient:

$$w_k = \frac{N_{\text{train}}}{K \cdot N_k} \tag{5}$$

B. Kiến trúc mạng CNN tự xây dựng (Custom CNN)

Kiến trúc mô hình gồm 4 khối tích chập tuần tự theo định hướng Network-In-Network [31], với số lượng kênh tăng dần qua mỗi khối: $32 \to 64 \to 128 \to 256$. Cấu trúc chuẩn của mỗi khối bao gồm một lớp tích chập (`Conv2D` 3×3, `padding='same'`, không dùng hệ số chệch `use_bias=False` do đã tích hợp tham số dịch chuyển β trong tầng chuẩn hóa kế tiếp), nối tiếp bởi `BatchNormalization` [9], hàm kích hoạt `ReLU` [32], và kết thúc bằng phép gộp cực đại `MaxPooling2D` (2×2).

Về phần phân loại đầu ra, tensor đặc trưng từ khối thứ 4 được nén gọn bằng lớp `GlobalAveragePooling2D` (GAP), sau đó đi qua tầng ẩn `Dense(128, ReLU)` và tầng phân loại cuối cùng `Dense(6, Softmax)`. Đặc biệt, trong giai đoạn tinh chỉnh nâng cao (Advanced), các lớp `Dropout` [33] với tỷ lệ $0,5$ (trước tầng `Dense` ẩn) và $0,25$ (trước tầng phân loại) được kích hoạt nhằm chống quá khớp. Toàn bộ kiến trúc chi tiết được tổng kết trong Bảng I với tổng số 1.199.462 tham số.

Bảng I. Cấu trúc mạng CNN tự xây dựng (đầu vào $224 \times 224 \times 3$)

Khối	Cấu trúc tầng	Kích thước đầu ra	Số tham số
1	Conv32, BN, ReLU, MaxPool	$112 \times 112 \times 32$	992
2	$2 \times$ (Conv64, BN, ReLU), MaxPool	$56 \times 56 \times 64$	55.808
3	$2 \times$ (Conv128, BN, ReLU), MaxPool	$28 \times 28 \times 128$	222.208
4	$2 \times$ (Conv256, BN, ReLU), MaxPool	$14 \times 14 \times 256$	885.760
–	GlobalAveragePooling2D (GAP)	256	0
Head	Dense128 (ReLU) + Dropout	128	32.896
Head	Dense6 (Softmax)	6	774
Tổng cộng (bao gồm cả tham số BN)			**1.199.462**

Lý giải toán học việc dùng GAP thay cho Flatten: Sau 4 lần gộp cực đại, bản đồ đặc trưng có kích thước $14 \times 14 \times 256 = 50.176$ phần tử. Nếu dùng `Flatten` rồi nối vào `Dense(128)`, riêng tầng này đã tiêu tốn $50.176 \times 128 + 128 = 6.422.656$ tham số, đẩy kích thước toàn mạng lên $\approx 7,59$ triệu tham số (gấp $6,3$ lần). Ngược lại, phép toán GAP tính trung bình không gian trên từng kênh:

$$g_c = \frac{1}{H' \times W'} \sum_{u=1}^{H'} \sum_{v=1}^{W'} F_{u,v,c}, \quad \forall c \in \{1, \ldots, 256\} \tag{6}$$

giúp nén tensor về vector 256 chiều, chỉ cần 32.896 tham số.

C. Mô hình MobileNetV2 và ResNet50

MobileNetV2 [20]: Dựa trên hai cơ chế: (i) Tích chập tách theo chiều sâu (*Depthwise Separable Convolution*) phân rã tích chập chuẩn thành tích chập chiều sâu 3×3 và tích chập điểm 1×1, giúp giảm chi phí tính toán theo tỷ lệ [19]:

$$\frac{\text{Cost}_{\text{SepConv}}}{\text{Cost}_{\text{StdConv}}} = \frac{1}{C_{\text{out}}} + \frac{1}{D_K^2} \approx \frac{1}{9} \quad (\text{với } D_K = 3) \tag{7}$$

tức giảm khoảng 8–9 lần phép nhân cộng; và (ii) Khối thặng dư đảo ngược cùng nút thắt cổ chai tuyến tính (*Inverted Residual & Linear Bottleneck*) mở rộng kênh bằng tích chập 1×1 (hệ số $t = 6$), trích xuất đặc trưng bằng depthwise 3×3, rồi chiếu về chiều thấp qua phép chiếu tuyến tính (loại bỏ ReLU ở tầng nén để bảo toàn thông tin).

ResNet50 [18]: Định hướng các khối tầng học hàm phần dư $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ thay vì ánh xạ trực tiếp, tạo nên kết nối tắt nhận dạng:

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{8}$$

Khi lan truyền ngược, đạo hàm theo đầu vào của khối được tách thành tổng hai thành phần:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathcal{H}} \left(1 + \frac{\partial \mathcal{F}(\mathbf{x})}{\partial \mathbf{x}}\right) \tag{9}$$

nhờ đó tín hiệu gradient truyền trực tiếp qua số hạng 1 về các tầng đầu mà không bị suy hao qua 16 khối Bottleneck ($1\times1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$). Bảng II thống kê phân bố tham số của hai mạng.

Bảng II. Phân bố tham số của hai mô hình học chuyển giao

Thành phần mạng	MobileNetV2	ResNet50
Backbone tiền huấn luyện (ImageNet)	2.257.984	23.587.712
Classification Head (Dense128 + Dense6)	164.742	263.046
Tổng số tham số	**2.422.726**	**23.850.758**
Tham số mở ở Bước B (30 lớp cuối, trừ BN)	1.510.720	14.428.672
Tỷ lệ tham số được tinh chỉnh ở Bước B	62,4%	60,5%

D. Tiền xử lý và Tăng cường dữ liệu thích nghi

Mỗi mô hình nhận thống nhất ảnh thang $[0, 255]$ và tự tiền xử lý bên trong bằng tầng `layers.Lambda(preprocess_fn)` để tránh lệch phân phối giữa lúc huấn luyện và khi triển khai thực tế (*training-serving skew*): MobileNetV2 đưa về $[-1, 1]$; ResNet50 chuyển sang BGR và trừ trung bình ImageNet; Custom CNN chia cho 255 về $[0, 1]$.

Khối tăng cường dữ liệu [34, 35] chạy trực tiếp trên GPU gồm: `RandomFlip` ngang/dọc, `RandomRotation(0.1)` ($\pm 36°$), `RandomZoom(0.15)`, `RandomTranslation(0.1, 0.1)` và `RandomContrast(0.2)`. Các lớp này chỉ kích hoạt khi `training=True` và tự động tắt khi thẩm định/kiểm thử. Chú ý không biến đổi sắc độ (Hue) vì màu nâu/thâm là dấu hiệu cốt lõi của quả hỏng.

E. Tinh chỉnh hai bước và Cơ chế khóa BatchNorm

Tầng BatchNorm [9] khi ở chế độ huấn luyện sẽ chuẩn hóa theo kỳ vọng $\mu_{\mathcal{B}}$ và phương sai $\sigma_{\mathcal{B}}^2$ của lô hiện tại theo hai bước tách biệt:

$$\widehat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \widehat{x}_i + \beta \tag{10}$$

đồng thời cập nhật thống kê trượt tích lũy:

$$\mu_{\text{running}} \leftarrow m\, \mu_{\text{running}} + (1 - m)\mu_{\mathcal{B}} \tag{11}$$

Với lô nhỏ ($B = 32$) và miền ảnh nông sản khác biệt với ImageNet, nếu để BatchNorm ở chế độ huấn luyện khi fine-tune sẽ ghi đè thống kê tích lũy đã học bằng ước lượng nhiễu, làm sụp đổ đặc trưng tiền huấn luyện. Do đó, backbone được cô lập thành sub-model luôn gọi với `base(x, training=False)` và khóa cứng `trainable=False` cho mọi tầng BatchNorm ngay cả ở Bước B (Thuật toán 1).

Thuật toán 1: Học chuyển giao hai bước chuẩn mực

Input: Tập $\mathcal{D}_{\text{train}}$, $\mathcal{D}_{\text{val}}$, khung xương b (ImageNet), trọng số lớp $\mathbf{w}$.
Output: Bộ trọng số tối ưu $\boldsymbol{\theta}^*$ có `val_loss` nhỏ nhất.

Khởi tạo kiến trúc: Aug → Preprocess → $b(\cdot, \texttt{training=False})$ → GAP → Dense128 → Softmax.
Pha 1 – Warm-up Head:
  Khóa toàn bộ backbone: $b.\texttt{trainable} \leftarrow \texttt{False}$.
  Biên dịch mô hình với Adam ($\eta = 10^{-3}$).
  Huấn luyện tối đa 10 epochs trên $\mathcal{D}_{\text{train}}$ (kèm `EarlyStopping`, `ReduceLROnPlateau`).
Pha 2 – Fine-tuning Top Layers:
  Mở khóa 30 tầng cuối của b (giữ `trainable=False`).
  Biên dịch lại mô hình với Adam ($\eta = 10^{-5}$).
  Huấn luyện tiếp 15 epochs với bộ Callbacks khởi tạo mới.
return $\boldsymbol{\theta}^*$

F. Thước đo đánh giá và Khoảng tin cậy Wilson

Với mỗi phân lớp $k \in \{0, \ldots, 5\}$, độ chính xác dương (Precision), độ nhạy (Recall) và điểm trung bình điều hòa Macro-F1 [26, 27] được xác định rõ ràng qua hệ công thức:

$$\text{Precision}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \qquad \text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} \tag{12}$$

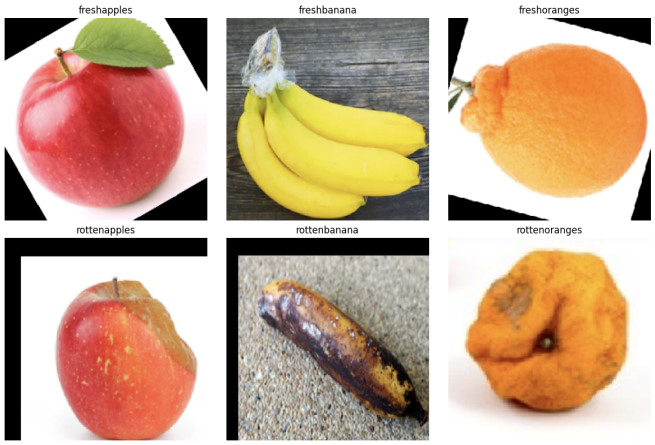
$$\text{Macro-F1} = \frac{1}{K} \sum_{k=0}^{K-1} \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{13}$$

Do độ chính xác kiểm thử $\widehat{p}$ trên $n = 2.698$ ảnh nằm sát biên trên 100%, xấp xỉ phân phối chuẩn truyền thống bị lệch. Sử dụng khoảng tin cậy nhị thức **Wilson 95%** ($z = 1,96$) [12] với giới hạn dưới và trên được tính theo phương trình:

$$\text{CI}_{\text{Wilson}} = \frac{\widehat{p} + \dfrac{z^2}{2n} \pm z\sqrt{\dfrac{\widehat{p}(1-\widehat{p})}{n} + \dfrac{z^2}{4n^2}}}{1 + \dfrac{z^2}{n}} \tag{14}$$

Trong đó, $\widehat{p}$ biểu diễn tỷ lệ mẫu phân loại chính xác, n là tổng số mẫu kiểm thử độc lập, và z là giá trị tới hạn tương ứng với mức ý nghĩa thống kê mong muốn (với độ tin cậy 95%, hệ số $z = 1,96$). Việc áp dụng khoảng tin cậy Wilson đóng vai trò then chốt trong việc cung cấp một cái nhìn khách quan, minh bạch và an toàn về giới hạn hiệu năng thực tế của mô hình trên tổng thể phân phối dữ liệu chưa từng thấy, qua đó giảm thiểu tối đa rủi ro khi triển khai hệ thống vào các dây chuyền phân loại nông sản tự động có yêu cầu khắt khe về độ tin cậy.

IV. DỮ LIỆU VÀ QUY TRÌNH XỬ LÝ



Hình 1. Mẫu ảnh trực quan đại diện cho 6 phân lớp nông sản tươi và hỏng trong tập huấn luyện.

A. Bộ dữ liệu và Phân bố lớp

Bộ dữ liệu *Fruits Fresh and Rotten for Classification* [36] gồm 13.599 ảnh (Hình 1). Thư mục `train` (10.901 ảnh) được tách độc lập 80%/20% thành tập Train (8.721 ảnh) và Validation (2.180 ảnh) với `SEED = 42`; thư mục `test` (2.698 ảnh) được giữ độc lập hoàn toàn để báo cáo cuối cùng. Bảng III trình bày phân bố mẫu và trọng số phạt w_k tính theo công thức (5). Tỷ lệ mất cân bằng trên tập Train là $1.859/1.181 \approx 1,57$, trong đó lớp thiểu số `freshoranges` được gán trọng số phạt cao nhất ($w = 1,231$).

Bảng III. Phân bố số lượng mẫu ảnh theo lớp và trọng số phạt w_k

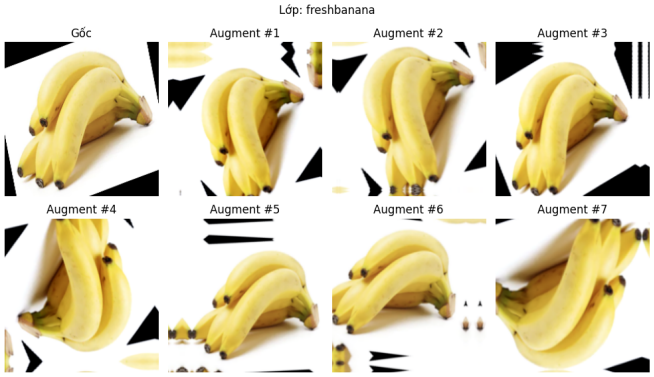
Lớp nông sản	Train	Val	Test	Tổng cộng	Trọng số w_k
`freshapples`	1.357	336	395	2.088	1,071
`freshbanana`	1.275	306	381	1.962	1,140
`freshoranges`	1.181	285	388	1.854	1,231
`rottenapples`	1.859	483	601	2.943	0,782
`rottenbanana`	1.767	457	530	2.754	0,823
`rottenoranges`	1.282	313	403	1.998	1,134
TỔNG CỘNG	**8.721**	**2.180**	**2.698**	**13.599**	–

B. Thiết kế Pipeline `tf.data` và Tối ưu hóa I/O

Luồng nạp dữ liệu TensorFlow [37] được cấu trúc tuần tự:

`Dataset` → `unbatch()[uint8]` → `cache()` → `shuffle(2048)` → `batch(32)[float32]` → `prefetch(AUTOTUNE)`

- **Giảm 4 lần dung lượng RAM:** Một ảnh $224 \times 224 \times 3$ chỉ chiếm 150.528 bytes ở kiểu `uint8` so với 602.112 bytes ở kiểu `float32`, toàn bộ 13.599 ảnh chỉ tốn $\approx 2,05$ GB.
- **Xáo trộn cấp độ từng ảnh:** Thực hiện `unbatch()` trước khi `cache()` và `shuffle(2048)` đảm bảo các mẫu trong mỗi lô được rút ngẫu nhiên qua từng epoch.
- **Chống rò rỉ và đồng bộ nhãn:** Kiểm tra tự động `assert set(train_paths).isdisjoint(val_paths)`, cố định thứ tự các lớp kiểm thử bằng tham số `class_names = train_raw.class_names` kèm `shuffle=False`.



Hình 2. Minh họa trực quan chuỗi biến đổi tăng cường dữ liệu thích nghi không gian trên GPU đối với mẫu ảnh `freshbanana`.

V. THIẾT LẬP THỰC NGHIỆM

Thực hiện 6 cấu hình thực nghiệm (3 kiến trúc $\times$ 2 pha) với hạt giống cố định `SEED = 42` trên môi trường TensorFlow 2.21.0 (GPU Google Colab), tóm tắt tại Bảng IV.

Bảng IV. Thiết lập siêu tham số cho 6 cấu hình thực nghiệm

Mô hình	Giai đoạn (Pha)	Epoch tối đa	Tốc độ học η
CustomCNN	Baseline (không Aug/Dropout)	5	10^{-3}
MobileNetV2	Baseline (khóa cứng backbone)	5	10^{-3}
ResNet50	Baseline (khóa cứng backbone)	5	10^{-3}
CustomCNN	Advanced (Aug + Dropout 0,5)	30	10^{-3}
MobileNetV2	Advanced: Bước A + Bước B	$10 + 15$	$10^{-3} \to 10^{-5}$
ResNet50	Advanced: Bước A + Bước B	$10 + 15$	$10^{-3} \to 10^{-5}$

Toàn bộ 6 cấu hình thực nghiệm trong nghiên cứu đều được tối ưu hóa đồng bộ bằng thuật toán học thích nghi **Adam** [38]. Quá trình cập nhật trọng số dựa trên việc tính toán ước lượng mô-men bậc một m_t đại diện cho kỳ vọng của gradient và mô-men bậc hai v_t đại diện cho kỳ vọng của bình phương gradient tại bước thời gian t, với các công thức thành phần được định nghĩa lần lượt như sau:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{15}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{16}$$

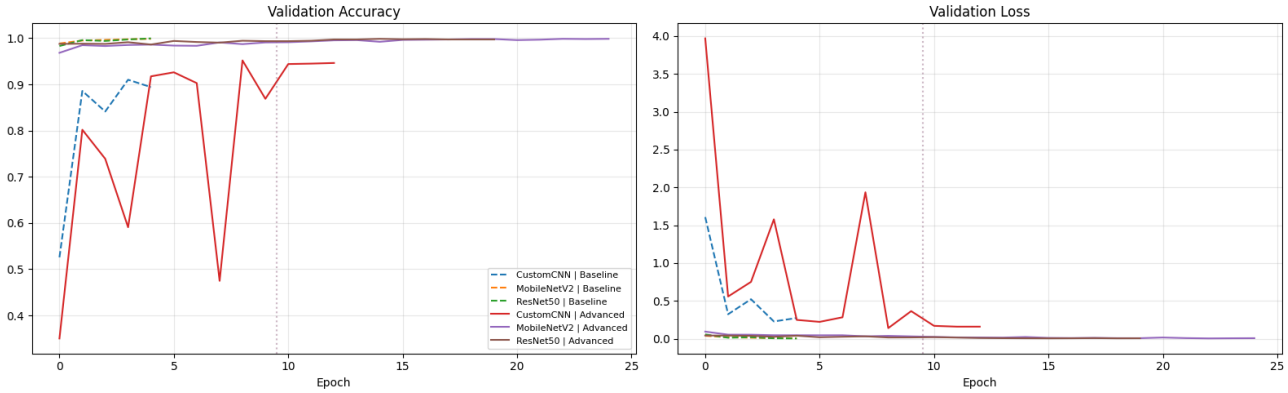
trong đó g_t là véc-tơ đạo hàm riêng của hàm mất mát tại thời điểm t. Để khắc phục hiện tượng chệch về gốc tọa độ trong các bước lặp ban đầu khi mô-men khởi tạo bằng không, quá trình hiệu chỉnh độ lệch được áp dụng qua các biểu thức:

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{17}$$

Cuối cùng, véc-tơ trọng số mô hình θ_t được cập nhật trực tiếp thông qua tốc độ học cơ sở η kết hợp với các giá trị mô-men đã được hiệu chỉnh:

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\widehat{v}_t} + \epsilon} \widehat{m}_t \tag{18}$$

thông qua thuật toán tối ưu hóa Adam, với các siêu tham số được thiết lập cố định bao gồm hệ số phân rã mô-men bậc nhất $\beta_1 = 0,9$, hệ số phân rã mô-men bậc hai $\beta_2 = 0,999$, cùng hằng số ổn định mẫu số nhằm chống chia cho không $\epsilon = 10^{-7}$ trong suốt quá trình huấn luyện mạng.



Hình 3. Đường cong học tập đối chứng độ chính xác thẩm định (Validation Accuracy – trái) và hàm mất mát thẩm định (Validation Loss – phải) của cả 6 mô hình qua các chu kỳ huấn luyện. Đường kẻ dọc nét đứt đánh dấu thời điểm kết thúc Bước A (Warm-up) và bắt đầu Bước B.

VI. PHÂN TÍCH KẾT QUẢ THỰC NGHIỆM

A. So sánh tổng hợp và Đường cong học tập

Bảng V kết quả tổng hợp cho thấy sự khác biệt rõ rệt giữa các mô hình qua hai pha thử nghiệm và khía cạnh triển khai thực tế, MobileNetV2 là giải pháp cân đối tối ưu nhất khi chỉ sở hữu khoảng $2,42$ triệu tham số nhưng vẫn duy trì được độ chính xác tương đương cao.

Hình 3 cho thấy hai mô hình học chuyển giao hội tụ tiệm cận trần chỉ sau 1–2 epochs nhờ tận dụng hiệu quả mạng không gian đặc trưng phong phú được huấn luyện trước trên tập dữ liệu ImageNet. Ngược lại, Custom CNN xuất hiện sự dao động mạnh ở những chu kỳ đầu của pha Advanced và chỉ bắt đầu đi vào trạng thái ổn định sau khi bộ điều tiết `ReduceLROnPlateau` thực hiện hạ hệ số học η theo các mức từ 10^{-3} xuống 2×10^{-4} và tiếp tục giảm còn 4×10^{-5}. Kết quả phân tích đối chứng chi tiết giữa hai pha huấn luyện đã khẳng định:

- **Custom CNN:** Ghi nhận mức tăng trưởng vượt bậc từ $89,29\% \rightarrow 96,00\%$. Điều này chứng minh vai trò mang tính sống còn của kỹ thuật tăng cường dữ liệu và cơ chế loại bỏ ngẫu nhiên trong quá trình huấn luyện các kiến trúc mạng từ đầu.
- **MobileNetV2 & ResNet50:** Khi các đặc trưng trích xuất từ ImageNet bị đóng băng, mô hình đã nhanh chóng đạt đến ngưỡng tiệm cận trần ($\approx 99,7\%$), khiến phần dư địa cải thiện còn lại trở nên nhỏ hơn sai số lấy mẫu ngẫu nhiên. Mặc dù vậy, ở Bước B tinh chỉnh trên MobileNetV2, việc giải phóng một số tầng vẫn giúp kéo độ mất mát kiểm thử giảm sâu từ $0,0073$ xuống $0,0042$, qua đó tối ưu hóa đáng kể độ tin cậy của các xác suất dự đoán đầu ra.

B. Khoảng cách tổng quát hóa và Chi phí huấn luyện

Bảng VI trình bày chi tiết chỉ số độ lệch tổng quát hóa được xác định bởi công thức $\text{Gap} = \text{Acc}_{\text{train}} - \text{Acc}_{\text{val}}$, cùng với việc đánh giá chi phí thời gian huấn luyện thực tế trên hạ tầng phần cứng thử nghiệm. Đáng chú ý, ở pha Advanced, giá trị $\text{Acc}_{\text{train}}$ được ghi nhận trong điều kiện các kỹ thuật Augmentation và Dropout đang bật liên tục nên có xu hướng thấp hơn so với Acc_{val}. Kết quả này không chỉ khẳng định bộ điều hòa đang hoạt động hết sức hiệu quả mà còn đảm bảo mô hình hoàn toàn không rơi vào trạng thái học tủ hay quá khớp.

C. Khoảng cách tổng quát hóa và Chi phí huấn luyện

Bảng VI phân tích chi tiết độ lệch tổng quát hóa $\text{Gap} = \text{Acc}_{\text{train}} - \text{Acc}_{\text{val}}$ cùng với thời gian huấn luyện thực tế trên hạ tầng. Đặc biệt, ở pha Advanced, giá trị $\text{Acc}_{\text{train}}$ được đo lường trong điều kiện các cơ chế Augmentation và Dropout đang được kích hoạt liên tục nên có xu hướng thấp hơn so với Acc_{val}. giúp mô hình kiểm soát tốt và hoàn toàn không xảy ra hiện tượng quá khớp.

Bảng VI. Khoảng cách tổng quát hóa và Thời gian huấn luyện trên GPU Colab

Mô hình	Pha	Train	Val	Gap	Epochs	Time
CustomCNN	Baseline	92,45%	89,40%	+3,05%	5	187s
MobileNetV2	Baseline	100,0%	99,86%	+0,14%	5	71s
ResNet50	Baseline	100,0%	99,95%	+0,05%	5	191s
CustomCNN	Advanced	93,33%	94,63%	-1,30%	13	760s
MobileNetV2	Advanced	99,58%	99,86%	-0,29%	$10 + 15$	534s
ResNet50	Advanced	99,90%	99,72%	+0,17%	$10 + 10$	1.080s

D. Đánh giá chuyên sâu trên tập Test

Bảng VII trình bày chi tiết hiệu năng của **ResNet50 | Baseline** trên tập kiểm thử độc lập với 2.698 mẫu. Kết quả cho thấy mọi phân lớp đều đạt `F1-Score` $\geq 0,9950$. Với độ chính xác tổng quát đạt $0,9974$ cùng các chỉ số trung bình toàn cục xuất sắc, mô hình thể hiện khả năng phân loại cực kỳ ổn định và đáng tin cậy trên thực tế.

Bảng VII. Báo cáo phân loại chi tiết (Classification Report) trên tập Test độc lập của ResNet50 | Baseline

Lớp nông sản	Precision	Recall	F1-Score	Số lượng
`freshapples`	0,9925	1,0000	0,9962	395
`freshbanana`	1,0000	1,0000	1,0000	381
`freshoranges`	0,9949	0,9974	0,9961	388
`rottenapples`	0,9967	0,9967	0,9967	601
`rottenbanana`	1,0000	1,0000	1,0000	530
`rottenoranges`	1,0000	0,9901	0,9950	403
Accuracy	–	–	**0,9974**	**2.698**
Macro Avg	**0,9973**	**0,9974**	**0,9973**	**2.698**
Weighted Avg	**0,9974**	**0,9974**	**0,9974**	**2.698**

Bảng V. Kết quả tổng hợp 6 cấu hình thực nghiệm trên tập Validation ($n = 2.180$) và tập Test độc lập ($n = 2.698$)

Mô hình	Pha	Tham số	Val Acc	Val Loss	Test Acc	CI 95% (Wilson)	Số ảnh sai / Macro-F1
CustomCNN	Baseline	1.199.462	89,40%	0,2741	89,29%	88,07–90,40%	289 ảnh / 0,8983
MobileNetV2	Baseline	2.422.726	99,86%	0,0073	99,67%	99,37–99,82%	9 ảnh / 0,9968
ResNet50	**Baseline**	**23.850.758**	**99,95%**	**0,0030**	**99,74%**	**99,47–99,87%**	**7 ảnh / 0,9973**
CustomCNN	Advanced	1.199.462	95,18%	0,1405	96,00%	95,19–96,67%	108 ảnh / 0,9605
MobileNetV2	Advanced	2.422.726	99,86%	0,0042	99,67%	99,37–99,82%	9 ảnh / 0,9965
ResNet50	Advanced	23.850.758	99,77%	0,0052	99,63%	99,32–99,80%	10 ảnh / 0,9963

Phân tích 7 hình ảnh nhầm lẫn: (1) `rottenoranges` → `rottenapples` (2 ảnh): cam hỏng xuất hiện các đốm thâm nâu lớn có hình thái rất tương đồng với vùng vỏ táo thối; (2) `rottenoranges` → `freshoranges` (2 ảnh): cam mới chớ hỏng nhẹ ở phần rìa khuất, trong khi mặt chính diện vẫn giữ nguyên màu cam tươi tiêu chuẩn; (3) `rottenapples` → `freshapples` (2 ảnh): táo bị thối sâu tại vùng hõm cuống nhưng lại bị che khuất góc nhìn của cảm biến; và (4) `freshoranges` → `freshapples` (1 ảnh): quả cam có sắc vỏ đỏ đặc trưng gần giống với giống táo đỏ thương mại.

Góc nhìn nghiệp vụ: Nếu quy đổi bài toán về dạng nhị phân theo trạng thái độ tươi của nông sản, thực tế chỉ có 4/7 trường hợp là nhầm lẫn trực tiếp về trạng thái chất lượng với. Cụ thể hơn, trong tổng số 1.534 mẫu ảnh quả hỏng kiểm định, chỉ ghi nhận vỏn vẹn 4 ảnh bị lọt lưới, tương đương tỷ lệ lọt quả hỏng cực thấp chỉ $0,26\%$; đồng thời trong 1.164 mẫu ảnh quả tươi, hoàn toàn không có bất kỳ trường hợp nào bị phân loại nhầm, qua đó đảm bảo tuyệt đối sự an toàn, không gây thất thoát nông sản hay ảnh hưởng tiêu cực đến chuỗi cung ứng thương mại.

E. Đánh đổi độ chính xác và Chi phí triển khai

Bảng VIII đặt hiệu năng cạnh chi phí phần cứng. Đáng chú ý, Custom CNN dù chỉ có $1,20$ triệu tham số nhưng tiêu tốn tới $\approx 2,12$ tỷ phép toán mỗi ảnh do các tầng tích chập đầu hoạt động trên bản đồ đặc trưng kích thước lớn, nặng gấp 7 lần MobileNetV2 [20]. Do MobileNetV2 chỉ kém ResNet50 đúng 2 ảnh trên tập Test nhưng nhẹ hơn 10 lần và suy luận nhanh gấp 3,2 lần trên CPU với tốc độ 14 ms/ảnh, Chọn ResNet50 làm mô hình quán quân lý thuyết và đề xuất MobileNetV2 cho triển khai thực tế trên thiết bị biên.

Bảng VIII. So sánh chi phí triển khai và suy luận thực tế trên CPU

Mô hình	Tham số	Kích thước	Phép tính	Độ trễ CPU
CustomCNN	1,20 M	4,8 MB	2,12 G	28 ms/ảnh
MobileNetV2	2,42 M	9,5 MB	0,30 G	14 ms/ảnh
ResNet50	23,85 M	95,0 MB	3,80 G	45 ms/ảnh

VII. THẢO LUẬN VÀ HẠN CHẾ

- **Ý nghĩa thống kê và kiểm định:** Khoảng tin cậy Wilson 95% giữa cấu hình ResNet50 và MobileNetV2 có vùng chồng lấn gần như hoàn toàn, cho thấy hiệu năng tổng quát của hai mô hình là rất tương đồng. Để có cơ sở khẳng định sự vượt trội có ý nghĩa thống kê thực sự bắt buộc phải áp dụng kiểm định phi tham số ghép cặp McNemar [39] kết hợp quy trình lặp thực nghiệm qua nhiều hạt giống ngẫu nhiên khác nhau.
- **Biến thể ảnh gần trùng lặp trong bộ dữ liệu nguồn:** Việc sử dụng hàm kiểm tra tập hợp `isdisjoint` trong tiền xử lý chỉ đảm bảo không bị trùng lặp về đường dẫn tệp tin. Tuy nhiên, qua quá trình quan sát thực tế tại Hình 1, tập dữ liệu gốc trên Kaggle đã chứa sẵn nhiều biến thể ảnh có cùng nguồn gốc nhưng bị xoay, dịch chuyển hoặc can thiệp viền đen. Do đó, cần tích hợp thuật toán băm tri giác để tự động phát hiện và gom nhóm các ảnh gốc trước khi tiến hành phân chia tập dữ liệu nhằm thực hiện đánh giá một cách khắt khe và khách quan nhất [8].
- **Miền dữ liệu và Hiện tượng hiệu chỉnh xác suất:** Bộ dữ liệu hiện tại bao gồm ba loại nông sản được chụp trên nền tương đối sạch và đồng nhất; điều này đặt ra rủi ro mô hình có thể rơi vào hiện tượng học tắt theo đặc trưng nền thay vì tập trung vào đặc tính thực của quả [28, 29]. Việc kiểm chứng khả năng chống chịu nhiễu môi trường, kết hợp đánh giá độ tin cậy và hiệu chỉnh xác suất của phân phối đầu ra Softmax [40] là cực kỳ cấp thiết trước khi đưa hệ thống vào vận hành thực tế trên các dây chuyền băng chuyền công nghiệp.

VIII. TRIỂN KHAI ỨNG DỤNG THỰC TẾ

Hệ thống Web Demo được đóng gói theo mô hình kiến trúc Client-Server tiêu chuẩn tại thư mục mã nguồn `web_demo`:

- **Backend (Flask RESTful API):** Cung cấp một endpoint chuyên biệt `/predict`, tiếp nhận ảnh thông qua định dạng `multipart/form-data`, tự động chuyển đổi sang không gian màu RGB bằng thư viện `PIL.Image`, thực hiện nội suy kích thước về chuẩn 224×224, truyền thẳng tensor thang $[0, 255]$ qua mô hình và trả về đối tượng JSON hoàn chỉnh gồm nhãn dự đoán, độ tự tin dưới dạng phần trăm (%). Toàn bộ cơ chế xử lý ngoại lệ, kiểm tra định dạng tệp tin và quản lý luồng dữ liệu vào ra cũng được tích hợp chặt chẽ nhằm đảm bảo tính ổn định tối đa cho máy chủ khi tiếp nhận đồng thời nhiều yêu cầu từ người dùng.
- **Frontend (HTML/CSS/JS):** Hỗ trợ tính năng kéo thả ảnh linh hoạt, hiển thị xem trước tức thì, tự động điều chỉnh tông màu cảnh báo thông minh với sắc xanh lá đặc trưng cho nhóm nhãn `Fresh` và sắc đỏ cho nhóm nhãn `Rotten`, kết hợp tích hợp thanh tiến trình trực quan hóa tường minh xác suất của từng phân lớp. Giao diện được thiết kế theo phong cách tối giản, hiện đại, đảm bảo tính tương thích mượt mà trên đa dạng các thiết bị nhằm mang lại trải nghiệm tương tác trực quan, thân thiện và hiệu quả nhất trong quá trình vận hành thực tế.

IX. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Bài báo đã xây dựng và đối chứng toàn diện 6 cấu hình phân loại trái cây tươi/hỏng trên 13.599 ảnh với pipeline dữ liệu `tf.data` chống rò rỉ, kiến trúc GAP triệt tiêu bùng nổ tham số và quy trình tinh chỉnh 2 bước bảo toàn thống kê BatchNorm. ResNet50 đạt Test Accuracy 99, 74% (Macro-F1 0, 9973), trong khi MobileNetV2 đạt 99, 67% với chi phí tính toán thấp hơn vượt trội và tốc độ suy luận 14 ms/ảnh trên CPU. Tỷ lệ lọt quả hỏng chỉ ở mức 0, 26% và không loại nhầm bất kỳ quả tươi nào.

Hướng phát triển tiếp theo:

1. Mở rộng sang bài toán Phát hiện đối tượng với kiến trúc YOLO [41] để định vị và đếm đồng thời nhiều quả trên băng chuyền liên tục theo thời gian thực.
2. Tích hợp kỹ thuật giải thích mô hình Grad-CAM [42] trực tiếp lên giao diện Web để hiển thị bản đồ nhiệt vùng đã bị nấm mốc/hoại tử.
3. Lượng tử hóa mô hình sang số nguyên 8-bit (INT8 Quantization) qua TensorFlow Lite [43] giúp thu gọn kích thước mô hình đáng kể mà vẫn duy trì độ chính xác, phù hợp để chạy thời gian thực trên bo mạch Raspberry Pi.
4. Khử trùng lặp bằng băm tri giác, kiểm định McNemar đa hạt giống và mở rộng sang kiến trúc EfficientNet [21].
5. Tích hợp cơ chế tự động học tăng cường trực tuyến nhằm cho phép hệ thống tự động thu thập các mẫu nhầm lẫn từ thực địa để cập nhật và tinh chỉnh trọng số liên tục.

Lời tuyên bố sử dụng AI

Em cam kết tính minh bạch và liêm chính học thuật về việc sử dụng các công cụ Trí tuệ nhân tạo trong quá trình nghiên cứu và biên soạn báo cáo theo danh mục kiểm tra dưới đây:

1. Sử dụng AI hỗ trợ viết báo cáo: **Yes**.
2. Sử dụng AI hỗ trợ sinh các biểu đồ: **No**.
3. Sử dụng AI hỗ trợ sinh giao diện: **Yes**.
4. Sử dụng AI hỗ trợ sinh test cases: **No**.
5. Sử dụng bao nhiêu % AI: **67%**.

REFERENCES

[1] J. Gustavsson, C. Cederberg, U. Sonesson, R. van Otterdijk, and A. Meybeck, *Global Food Losses and Food Waste: Extent, Causes and Prevention*. Rome, Italy: FAO, 2011.

[2] A. Bhargava and A. Bansal, "Fruits and vegetables quality evaluation using computer vision: A review," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 33, no. 3, pp. 243–257, 2021.

[3] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Comput. Electron. Agric.*, vol. 147, pp. 70–90, 2018, doi: https://doi.org/10.1016/j.compag.2018.02.016.

[4] J. Naranjo-Torres *et al.*, "A review of convolutional neural network applied to fruit image processing," *Appl. Sci.*, vol. 10, no. 10, Art. no. 3443, 2020, doi: https://doi.org/10.3390/app10103443.

[5] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Netw.*, vol. 106, pp. 249–259, 2018, doi: https://doi.org/10.1016/j.neunet.2018.07.011.

[6] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.

[7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org/.

[8] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, Art. no. 15, 2012, doi: https://doi.org/10.1145/2382577.2382579.

[9] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2015, pp. 448–456.

[10] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[11] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006, doi: https://doi.org/10.1016/j.patrec.2005.10.010.

[12] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927, doi: https://doi.org/10.1080/01621459.1927.10502953.

[13] H. Mureşan and M. Oltean, "Fruit recognition from images using deep learning," *Acta Univ. Sapientiae, Inform.*, vol. 10, no. 1, pp. 26–42, 2018.

[14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[15] J. Deng *et al.*, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.

[16] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.

[17] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[19] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," *arXiv:1704.04861*, 2017.

[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.

[21] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[22] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[23] S. Kornblith, J. Shlens, and Q. V. Le, "Do better ImageNet models transfer better?" in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 2661–2671.

[24] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 9268–9277.

[25] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 2980–2988.

[26] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Inf. Process. Manag.*, vol. 45, no. 4, pp. 427–437, 2009.

[27] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," *J. Mach. Learn. Technol.*, vol. 2, no. 1, pp. 37–63, 2011.

[28] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2011, pp. 1521–1528.

[29] R. Geirhos *et al.*, "Shortcut learning in deep neural networks," *Nat. Mach. Intell.*, vol. 2, no. 11, pp. 665–673, 2020.

[30] S. Raschka, "Model evaluation, model selection, and algorithm selection in machine learning," *arXiv:1811.12808*, 2018.

[31] M. Lin, Q. Chen, and S. Yan, "Network in network," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.

[32] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2010, pp. 807–814.

[33] N. Srivastava *et al.*, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[34] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *J. Big Data*, vol. 6, no. 1, Art. no. 60, 2019.

[35] F. Chollet, *Deep Learning with Python*, 2nd ed. Manning Publications, 2021.

[36] S. Raghavan, "Fruits fresh and rotten for classification," *Kaggle Datasets*, 2018. [Online]. Available: https://www.kaggle.com/datasets/sriramr/fruits-fresh-and-rotten-for-classification.

[37] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. USENIX OSDI*, 2016, pp. 265–283.

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[39] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.

[40] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.

[41] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[42] R. R. Selvaraju *et al.*, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.

[43] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 2704–2713.